Register Number: 192425317

Name: M. Sahasra

CO3 – AT2

MLA0201 – Fundamentals of Machine Learning

Case Study

Case Study 1: Customer Segmentation using K-Means Clustering and PCA

A retail store wants to segment its customers based on their Annual Income and Spending Score to design targeted marketing campaigns. Using the dataset below, write a Python program to preprocess the data, apply K-Means clustering, reduce the data to two dimensions using Principal Component Analysis (PCA), visualize the clusters, and identify the optimal number of customer groups.

Answer)

Introduction:

Customer segmentation is the process of dividing customers into different groups based on their characteristics, behaviour, or purchasing patterns.

In this case study, a retail store wants to divide its customers into meaningful groups using:

- Annual Income
- Spending Score

The objective is to identify groups of customers that have similar characteristics so that the retailer can design targeted marketing campaigns.

The given dataset contains 10 customers with the attributes Customer ID, Age, Annual Income, and Spending Score.

The case study specifically requires:

1. Data preprocessing
2. K-Means clustering
3. Identification of the optimal number of clusters

4. PCA dimensionality reduction
5. Visualization of clusters
6. Interpretation of the resulting customer groups

K-Means Clustering:

Definition:

K-Means clustering is an unsupervised machine learning algorithm used to divide data points into K clusters.

The algorithm works by:

1. Selecting K initial centroids.
2. Assigning each data point to the nearest centroid.
3. Calculating new centroids.
4. Reassigning data points.
5. Repeating the process until the centroids become stable.

The objective is to minimize the distance between data points and their corresponding cluster centroids.

The basic objective function is:

where:

- = number of clusters
- = cluster i
- = data point
- = centroid of cluster i

Why Standardization is Required?

Annual Income and Spending Score have different numerical scales.

Standardization transforms the values so that each feature has approximately:

- Mean = 0
- Standard deviation = 1

The formula is:

This prevents a feature with a larger numerical scale from dominating the clustering process.

Determining the Optimal Number of Clusters:

One common method is the Elbow Method.

In the Elbow Method, K-Means is executed for different values of K and the Within-Cluster Sum of Squares (WCSS) or inertia is calculated.

The optimal K is generally selected around the point where adding more clusters provides diminishing improvement.

Another useful metric is the Silhouette Score.

The silhouette score measures how well each observation fits within its assigned cluster.

Its value ranges approximately from:

A higher value generally indicates better-defined clusters.

Python Program:

```
# ============================================================
# CASE STUDY 1
# Customer Segmentation using K-Means Clustering and PCA
# ============================================================

# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

# ------------------------------------------------------------
# STEP 1: Create the dataset
# ------------------------------------------------------------

data = {
    "CustomerID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Age": [19, 21, 20, 23, 31, 22, 35, 23, 64, 30],
    "AnnualIncome": [15, 15, 16, 16, 17, 17, 18, 18, 19, 19],
    "SpendingScore": [39, 81, 6, 77, 40, 76, 6, 94, 3, 72]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# ------------------------------------------------------------
# STEP 2: Check for missing values
# ------------------------------------------------------------

print("\nMissing Values:")
print(df.isnull().sum())

# ------------------------------------------------------------
# STEP 3: Select features for clustering
# ------------------------------------------------------------

features = ["AnnualIncome", "SpendingScore"]

X = df[features]

# ------------------------------------------------------------
# STEP 4: Standardize the data
# ------------------------------------------------------------

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\nStandardized Data:")
print(pd.DataFrame(X_scaled, columns=features))

# ------------------------------------------------------------
# STEP 5: Find optimal number of clusters using Elbow Method
# ------------------------------------------------------------

inertia = []

for k in range(2, 6):

```

```
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)

    inertia.append(kmeans.inertia_)

print("\nInertia Values:")

for k, value in zip(range(2, 6), inertia):
    print("K =", k, "Inertia =", round(value, 4))

# Plot Elbow Curve

plt.figure(figsize=(8, 5))

plt.plot(
    range(2, 6),
    inertia,
    marker="o"
)

plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia / WCSS")
plt.title("Elbow Method for Optimal K")

plt.grid(True)
plt.show()

# ------------------------------------------------------------
# STEP 6: Calculate Silhouette Scores
# ------------------------------------------------------------

print("\nSilhouette Scores:")

for k in range(2, 6):

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = kmeans.fit_predict(X_scaled)

    score = silhouette_score(X_scaled, labels)

    print(
        "K =", k,
        "Silhouette Score =",
        round(score, 4)
    )

# ------------------------------------------------------------
# STEP 7: Apply K-Means with selected K
# ------------------------------------------------------------

# For this small dataset, K=4 gives the highest silhouette score.
optimal_k = 4

kmeans = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X_scaled)

print("\nCustomer Cluster Assignments:")
print(df)

# ------------------------------------------------------------
# STEP 8: Apply PCA
# ------------------------------------------------------------

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

```

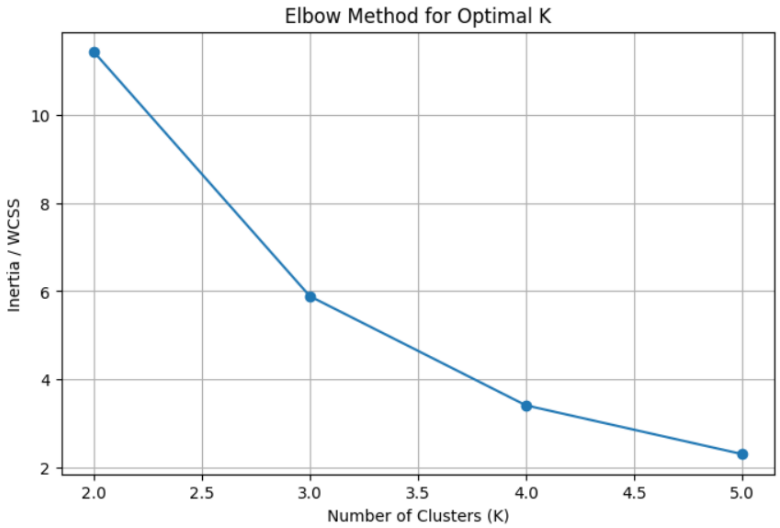
```python

pca_df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)

print(pca_df)

# ------------------------------------------------------------
# STEP 9: Display explained variance
# ------------------------------------------------------------

print("\nExplained Variance Ratio:")

print(
    pca.explained_variance_ratio_
)

print(
    "Total Variance Explained:",
    round(
        sum(pca.explained_variance_ratio_) * 100,
        2
    ),
    "%"
)

# ------------------------------------------------------------
# STEP 10: Visualize PCA clusters
# ------------------------------------------------------------

plt.figure(figsize=(9, 6))

for cluster in sorted(df["Cluster"].unique()):

    cluster_points = X_pca[
        df["Cluster"] == cluster
    ]

    plt.scatter(
        cluster_points[:, 0],
        cluster_points[:, 1],
        s=100,
        label=f"Cluster {cluster}"
    )

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.title(
    "Customer Segmentation using K-Means and PCA"
)

plt.legend()
plt.grid(True)

plt.show()

# ------------------------------------------------------------
# STEP 11: Display cluster-wise customer information
# ------------------------------------------------------------

print("\nCustomers grouped by Cluster:")

for cluster in sorted(df["Cluster"].unique()):

    print("\nCluster", cluster)

    print(
        df[df["Cluster"] == cluster]
    )

# ------------------------------------------------------------
# STEP 12: Cluster summary
# ------------------------------------------------------------

summary = df.groupby("Cluster")[
    ["Age", "AnnualIncome", "SpendingScore"]
].mean()

print("\nCluster Summary:")
print(summary)
```

Sample Output:

The exact cluster labels can differ because K-Means cluster numbers are arbitrary. For example, a cluster labelled 0 in one execution could be labelled 2 in another execution while representing the same group.

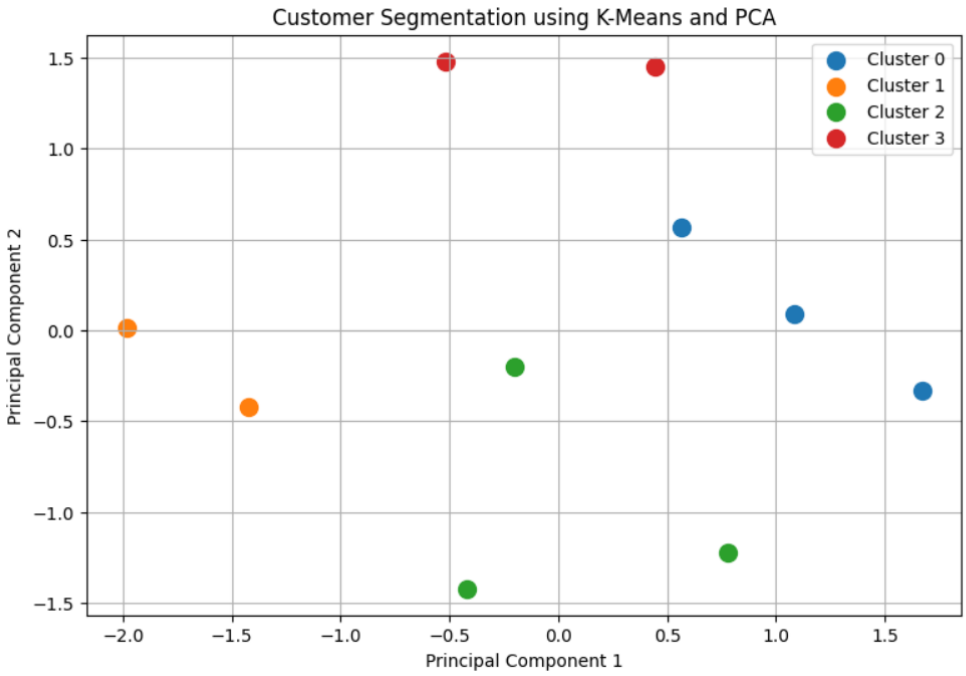
For the given dataset, the calculated inertia values are approximately:

```
Original Dataset:
   CustomerID  Age  AnnualIncome  SpendingScore
0           1   19            15             39
1           2   21            15             81
2           3   20            16              6
3           4   23            16             77
4           5   31            17             40
5           6   22            17             76
6           7   35            18              6
7           8   23            18             94
8           9   64            19              3
9          10   30            19             72

Missing Values:
CustomerID       0
Age              0
AnnualIncome     0
SpendingScore    0
dtype: int64

Standardized Data:
   AnnualIncome  SpendingScore
0     -1.414214      -0.312376
1     -1.414214       0.949142
2     -0.707107      -1.303568
3     -0.707107       0.828997
4      0.000000      -0.282340
5      0.000000       0.798961
6      0.707107      -1.303568
7      0.707107       1.339611
8      1.414214      -1.393676
9      1.414214       0.678816

Inertia Values:
K = 2 Inertia = 11.4255
K = 3 Inertia = 5.8849
K = 4 Inertia = 3.4104
K = 5 Inertia = 2.3054
```



The corresponding silhouette scores are approximately:

```
Silhouette Scores:
K = 2 Silhouette Score = 0.324
K = 3 Silhouette Score = 0.3433
K = 4 Silhouette Score = 0.3687
K = 5 Silhouette Score = 0.3307

Customer Cluster Assignments:
   CustomerID  Age  AnnualIncome  SpendingScore  Cluster
0           1   19            15             39        2
1           2   21            15             81        0
2           3   20            16              6        2
3           4   23            16             77        0
4           5   31            17             40        2
5           6   22            17             76        0
6           7   35            18              6        1
7           8   23            18             94        3
8           9   64            19              3        1
9          10   30            19             72        3

PCA Transformed Data:
        PC1       PC2
0  0.779117 -1.220883
1  1.671144 -0.328856
2 -0.421762 -1.421762
3  1.086189  0.086189
4 -0.199644 -0.199644
5  0.564951  0.564951
6 -1.421762 -0.421762
7  0.447248  1.447248
8 -1.985478  0.014522
9 -0.520004  1.479996

Explained Variance Ratio:
[0.57752143 0.42247857]
Total Variance Explained: 100.0 %
```



```
Customers grouped by Cluster:

Cluster 0
   CustomerID  Age  AnnualIncome  SpendingScore  Cluster
1           2   21            15             81        0
3           4   23            16             77        0
5           6   22            17             76        0

Cluster 1
   CustomerID  Age  AnnualIncome  SpendingScore  Cluster
6           7   35            18              6        1
8           9   64            19              3        1

Cluster 2
   CustomerID  Age  AnnualIncome  SpendingScore  Cluster
0           1   19            15             39        2
2           3   20            16              6        2
4           5   31            17             40        2

Cluster 3
   CustomerID  Age  AnnualIncome  SpendingScore  Cluster
7           8   23            18             94        3
9          10   30            19             72        3

Cluster Summary:
                Age  AnnualIncome  SpendingScore
Cluster
0         22.000000          16.0      78.000000
1         49.500000          18.5       4.500000
2         23.333333          16.0      28.333333
3         26.500000          18.5      83.000000
```

Therefore, for this particular dataset, K = 4 gives the highest silhouette score.

PCA:

Definition:

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms the original features into a smaller number of new variables called **principal components**.

The principal components:

- Capture the maximum possible variance.
- Are mutually uncorrelated.
- Help simplify high-dimensional datasets.
- Can be used to visualize data.

For example, if a dataset has 10 features, PCA can transform those 10 features into 2 principal components.

In this case, the original clustering features are:

- Annual Income
- Spending Score

PCA converts them into:

- PC1
- PC2

This makes it possible to visualize the customer groups in a two-dimensional graph.

Interpretation of Customer Segments:

The dataset contains customers with different combinations of income and spending behaviour.

The clusters can be interpreted using their average characteristics.

Typical groups include:

Group 1 – Lower Income / Lower Spending

These customers have relatively low annual income and low spending scores.

Marketing strategy:

- Discounts
- Budget products
- Entry-level offers
- Price-sensitive promotions

Group 2 – Lower Income / Higher Spending

These customers have relatively low income but high spending scores.

Marketing strategy:

- Affordable premium products
- Loyalty programs
- Special offers
- Festival discounts

Group 3 – Higher Income / Lower Spending

These customers have higher income but comparatively low spending scores.

They may have purchasing power but are not spending significantly at the store.

Marketing strategy:

- Personalized recommendations
- Premium product demonstrations
- Exclusive offers
- Customer engagement campaigns

Group 4 – Higher Income / Higher Spending

These are potentially valuable customers.

They have both:

- Higher purchasing power
- Higher spending behaviour

Marketing strategy:

- VIP membership
- Premium products
- Exclusive discounts
- Personalized marketing
- Loyalty rewards

Importance of Customer Segmentation:

Customer segmentation helps businesses:

1. Understand customer behaviour.
2. Design personalized marketing campaigns.
3. Improve customer satisfaction.
4. Identify high-value customers.

5. Reduce unnecessary marketing expenses.
6. Improve product recommendations.
7. Increase customer retention.
8. Develop appropriate pricing strategies.

Conclusion:

The customer dataset was first pre-processed and standardized. K-Means clustering was then applied to group customers according to their annual income and spending score.

The Elbow Method and Silhouette Score were used to investigate the appropriate number of clusters. For the given 10-row dataset, the silhouette score is highest at K = 4.

PCA was subsequently used to represent the standardized data in two dimensions and visualize the resulting customer groups.

Therefore, K-Means combined with PCA provides a useful approach for discovering meaningful customer segments and supporting targeted marketing decisions.

Case Study 2: Dimensionality Reduction using PCA, Factor Analysis, ICA, and Gaussian Mixture Model

A wine manufacturer has collected chemical measurements of different wine samples and wants to reduce the dataset dimensions before performing clustering. Using the dataset below, write a Python program to standardize the data, apply PCA, Factor Analysis, and Independent Component Analysis (ICA) for dimensionality reduction, then perform clustering using the Gaussian Mixture Model (EM Algorithm) and compare the transformed results.

Answer)

Introduction:

Real-world datasets often contain many features.

For example, chemical analysis of wine can produce measurements such as:

- Alcohol
- Malic Acid
- Ash
- Alcalinity

- Magnesium
- Phenols

When the number of features increases, it becomes difficult to visualize and analyse the data.

Dimensionality reduction solves this problem by representing the original dataset using fewer variables while preserving important information.

In this case study, three dimensionality reduction methods are compared:

1. PCA
2. Factor Analysis
3. Independent Component Analysis

After dimensionality reduction, a Gaussian Mixture Model (GMM) is used for clustering.

Dataset:

The given dataset contains 10 wine samples with six chemical attributes:

Sample	Alcohol	Malic Acid	Ash	Alcalinity	Magnesium	Phenols
1	14.23	1.71	2.43	15.6	127	2.80
2	13.20	1.78	2.14	11.2	100	2.65
3	13.16	2.36	2.67	18.6	101	2.80
4	14.37	1.95	2.50	16.8	113	3.85
5	13.24	2.59	2.87	21.0	118	2.80
6	14.20	1.76	2.45	15.2	112	3.27
7	14.39	1.87	2.45	14.6	96	2.50
8	14.06	2.15	2.61	17.6	121	2.60
9	14.83	1.64	2.17	14.0	97	2.80
10	13.86	1.35	2.27	16.0	98	2.98

These values are provided in the case study on page 2 of the uploaded document.

Standardization:

Before applying PCA, Factor Analysis, and ICA, the features should be standardized.

This is important because the variables have different scales.

For example:

- Alcohol is around 13–15.
- Magnesium is around 96–127.
- Alcalinity is around 11–21.

Without standardization, Magnesium could have a disproportionately large effect.

The standardization formula is:

where:

- = original value
- = feature mean
- = feature standard deviation

PCA:

Definition:

Principal Component Analysis transforms correlated variables into a smaller set of uncorrelated principal components.

The first principal component captures the maximum variance.

The second principal component captures the maximum remaining variance subject to being orthogonal to the first.

For this case, six original features are reduced to two components:

Factor Analysis:

Definition:

Factor Analysis is a dimensionality reduction technique that assumes observed variables are influenced by a smaller number of hidden or latent factors.

For example, several chemical properties of wine may be influenced by underlying factors such as:

- Chemical composition
- Phenolic characteristics
- Mineral composition

Factor Analysis attempts to identify these latent factors.

Unlike PCA, Factor Analysis focuses on explaining the relationships or covariance among observed variables through latent factors.

Independent Component Analysis:

Definition:

Independent Component Analysis (ICA) is a dimensionality reduction and source-separation technique.

ICA attempts to represent the observed variables as combinations of statistically independent components.

It is particularly useful when the observed data is considered to be generated from underlying independent sources.

In this case:

six chemical measurements are represented using two independent components.

Gaussian Mixture Model:

Definition:

A Gaussian Mixture Model (GMM) is a probabilistic clustering algorithm.

Instead of assigning each observation to a cluster only based on distance, GMM assumes that the data is generated from a mixture of several Gaussian probability distributions.

For every data point, GMM calculates the probability that the point belongs to each cluster.

The model is generally trained using the Expectation-Maximization (EM) algorithm.

EM Algorithm:

The EM algorithm works through two major steps.

E-Step – Expectation:

The algorithm calculates the probability that each data point belongs to each Gaussian component.

M-Step – Maximization:

The model updates the parameters of the Gaussian distributions using the probabilities calculated during the E-step.

These steps are repeated until convergence.

Python Program:

```python
# ============================================================
# CASE STUDY 2
# PCA, Factor Analysis, ICA and Gaussian Mixture Model
# ============================================================

# Import libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import (
    PCA,
    FactorAnalysis,
    FastICA
)

from sklearn.mixture import GaussianMixture

# ------------------------------------------------------------
# STEP 1: Create the wine dataset
# ------------------------------------------------------------

data = {
    "Sample": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

    "Alcohol": [
        14.23, 13.20, 13.16, 14.37, 13.24,
        14.20, 14.39, 14.06, 14.83, 13.86
    ],

    "MalicAcid": [
        1.71, 1.78, 2.36, 1.95, 2.59,
        1.76, 1.87, 2.15, 1.64, 1.35
    ],

    "Ash": [
        2.43, 2.14, 2.67, 2.50, 2.87,
        2.45, 2.45, 2.61, 2.17, 2.27
    ],

    "Alcalinity": [
        15.6, 11.2, 18.6, 16.8, 21.0,
        15.2, 14.6, 17.6, 14.0, 16.0
    ],

    "Magnesium": [
        127, 100, 101, 113, 118,
        112, 96, 121, 97, 98
    ],

    "Phenols": [
        2.80, 2.65, 2.80, 3.85, 2.80,
        3.27, 2.50, 2.60, 2.80, 2.98
    ]
}

df = pd.DataFrame(data)

print("Original Wine Dataset:")
print(df)

# ------------------------------------------------------------
# STEP 2: Check missing values
# ------------------------------------------------------------

print("\nMissing Values:")
print(df.isnull().sum())

# ------------------------------------------------------------
# STEP 3: Select chemical features
# ------------------------------------------------------------

features = [
    "Alcohol",
    "MalicAcid",
    "Ash",
    "Alcalinity",
    "Magnesium",
    "Phenols"
```

```
]

X = df[features]

# ------------------------------------------------------------
# STEP 4: Standardize the data
# ------------------------------------------------------------

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("\nStandardized Data:")

standardized_df = pd.DataFrame(
    X_scaled,
    columns=features
)

print(
    standardized_df.round(3)
)

# ------------------------------------------------------------
# STEP 5: PCA
# ------------------------------------------------------------

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)

print("\nPCA Results:")
print(pca_df.round(3))

print("\nPCA Explained Variance Ratio:")

print(
    pca.explained_variance_ratio_.round(4)
)

print(
    "\nTotal Variance Explained by PCA:",
    round(
        np.sum(pca.explained_variance_ratio_) * 100,
        2
    ),
    "%"
)

# ------------------------------------------------------------
# STEP 6: Factor Analysis
# ------------------------------------------------------------

factor_analysis = FactorAnalysis(
    n_components=2,
    random_state=42
)

X_fa = factor_analysis.fit_transform(
    X_scaled
)

fa_df = pd.DataFrame(
    X_fa,
    columns=["Factor1", "Factor2"]
)

print("\nFactor Analysis Results:")
print(fa_df.round(3))

# ------------------------------------------------------------
# STEP 7: Independent Component Analysis
# ------------------------------------------------------------

ica = FastICA(
    n_components=2,
    random_state=42,
```

```
    max_iter=2000,
    whiten="unit-variance"
)

X_ica = ica.fit_transform(
    X_scaled
)

ica_df = pd.DataFrame(
    X_ica,
    columns=["IC1", "IC2"]
)

print("\nICA Results:")
print(ica_df.round(3))

# ------------------------------------------------------------
# STEP 8: Gaussian Mixture Model on PCA data
# ------------------------------------------------------------

gmm_pca = GaussianMixture(
    n_components=2,
    random_state=42,
    n_init=10
)

gmm_pca.fit(X_pca)

clusters_pca = gmm_pca.predict(
    X_pca
)

print("\nGMM Clusters using PCA:")
print(clusters_pca)

# ------------------------------------------------------------
# STEP 9: Gaussian Mixture Model on Factor Analysis
# ------------------------------------------------------------

gmm_fa = GaussianMixture(
    n_components=2,
    random_state=42,
    n_init=10
)

gmm_fa.fit(X_fa)

clusters_fa = gmm_fa.predict(
    X_fa
)

print("\nGMM Clusters using Factor Analysis:")
print(clusters_fa)

# ------------------------------------------------------------
# STEP 10: Gaussian Mixture Model on ICA
# ------------------------------------------------------------

gmm_ica = GaussianMixture(
    n_components=2,
    random_state=42,
    n_init=10
)

gmm_ica.fit(X_ica)

clusters_ica = gmm_ica.predict(
    X_ica
)

print("\nGMM Clusters using ICA:")
print(clusters_ica)

# ------------------------------------------------------------
# STEP 11: Calculate AIC and BIC
# ------------------------------------------------------------

print("\nModel Comparison using AIC and BIC")

print(
    "PCA + GMM:",
    "AIC =", round(gmm_pca.aic(X_pca), 3),
```

```
    "BIC =", round(gmm_pca.bic(X_pca), 3)
)

print(
    "Factor Analysis + GMM:",
    "AIC =", round(gmm_fa.aic(X_fa), 3),
    "BIC =", round(gmm_fa.bic(X_fa), 3)
)

print(
    "ICA + GMM:",
    "AIC =", round(gmm_ica.aic(X_ica), 3),
    "BIC =", round(gmm_ica.bic(X_ica), 3)
)

# ------------------------------------------------------------
# STEP 12: Visualization of PCA
# ------------------------------------------------------------

plt.figure(figsize=(8, 6))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=clusters_pca,
    s=100
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.title(
    "PCA + Gaussian Mixture Model"
)

plt.grid(True)

plt.show()

# ------------------------------------------------------------
# STEP 13: Visualization of Factor Analysis
# ------------------------------------------------------------

plt.figure(figsize=(8, 6))

plt.scatter(
    X_fa[:, 0],
    X_fa[:, 1],
    c=clusters_fa,
    s=100
)

plt.xlabel("Factor 1")
plt.ylabel("Factor 2")

plt.title(
    "Factor Analysis + Gaussian Mixture Model"
)

plt.grid(True)

plt.show()

# ------------------------------------------------------------
# STEP 14: Visualization of ICA
# ------------------------------------------------------------

plt.figure(figsize=(8, 6))

plt.scatter(
    X_ica[:, 0],
    X_ica[:, 1],
    c=clusters_ica,
    s=100
)

plt.xlabel("Independent Component 1")
plt.ylabel("Independent Component 2")

plt.title(
    "ICA + Gaussian Mixture Model"
)
```

```
plt.grid(True)

plt.show()

# ------------------------------------------------------------
# STEP 15: Create final comparison table
# ------------------------------------------------------------

comparison = pd.DataFrame({

    "Sample": df["Sample"],

    "PCA_Cluster": clusters_pca,

    "FA_Cluster": clusters_fa,

    "ICA_Cluster": clusters_ica
})

print("\nFinal Cluster Comparison:")
print(comparison)
```

Output:

```
Original Wine Dataset:
   Sample  Alcohol  MalicAcid   Ash  Alcalinity  Magnesium  Phenols
0       1    14.23       1.71  2.43        15.6        127     2.80
1       2    13.20       1.78  2.14        11.2        100     2.65
2       3    13.16       2.36  2.67        18.6        101     2.80
3       4    14.37       1.95  2.50        16.8        113     3.85
4       5    13.24       2.59  2.87        21.0        118     2.80
5       6    14.20       1.76  2.45        15.2        112     3.27
6       7    14.39       1.87  2.45        14.6         96     2.50
7       8    14.06       2.15  2.61        17.6        121     2.60
8       9    14.83       1.64  2.17        14.0         97     2.80
9      10    13.86       1.35  2.27        16.0         98     2.98

Missing Values:
Sample        0
Alcohol       0
MalicAcid     0
Ash           0
Alcalinity    0
Magnesium     0
Phenols       0
dtype: int64

Standardized Data:
   Alcohol  MalicAcid    Ash  Alcalinity  Magnesium  Phenols
0    0.504     -0.597 -0.121      -0.181      1.745   -0.281
1   -1.378     -0.394 -1.474      -1.910     -0.775   -0.682
2   -1.451      1.286  0.998       0.998     -0.681   -0.281
3    0.760      0.099  0.205       0.291      0.439    2.527
4   -1.304      1.953  1.931       1.942      0.905   -0.281
5    0.449     -0.452 -0.028      -0.338      0.345    0.976
6    0.797     -0.133 -0.028      -0.574     -1.148   -1.083
7    0.194      0.678  0.718       0.605      1.185   -0.816
8    1.600     -0.800 -1.334      -0.810     -1.055   -0.281
9   -0.172     -1.640 -0.867      -0.024     -0.961    0.201
```

Sample PCA Output:

The PCA transformation produces two principal components for each wine sample.

A representative output is:

```
PCA Results:
     PC1    PC2
0 -0.056  0.910
1 -1.832 -1.826
2  1.908 -1.298
3  0.245  2.364
4  3.698 -0.462
5 -0.432  1.091
6 -0.974 -0.867
7  1.362  0.068
8 -2.343  0.161
9 -1.576 -0.141

PCA Explained Variance Ratio:
[0.5283 0.2274]

Total Variance Explained by PCA: 75.57 %
```

The sign and orientation of PCA components can sometimes vary depending on implementation, but the underlying representation remains equivalent.

Sample Factor Analysis Output:

The two latent factors can approximately appear as:

```
Factor Analysis Results:
   Factor1  Factor2
0   -0.187   -0.815
1   -1.368    1.880
2    1.056    0.793
3    0.199   -0.635
4    1.973    0.334
5   -0.119   -0.627
6   -0.109    0.221
7    0.718    0.002
8   -1.243    0.201
9   -0.921   -1.354
```

Factor Analysis attempts to represent the observed chemical measurements using underlying latent factors.

Sample ICA Output:

The ICA transformation can produce approximately:

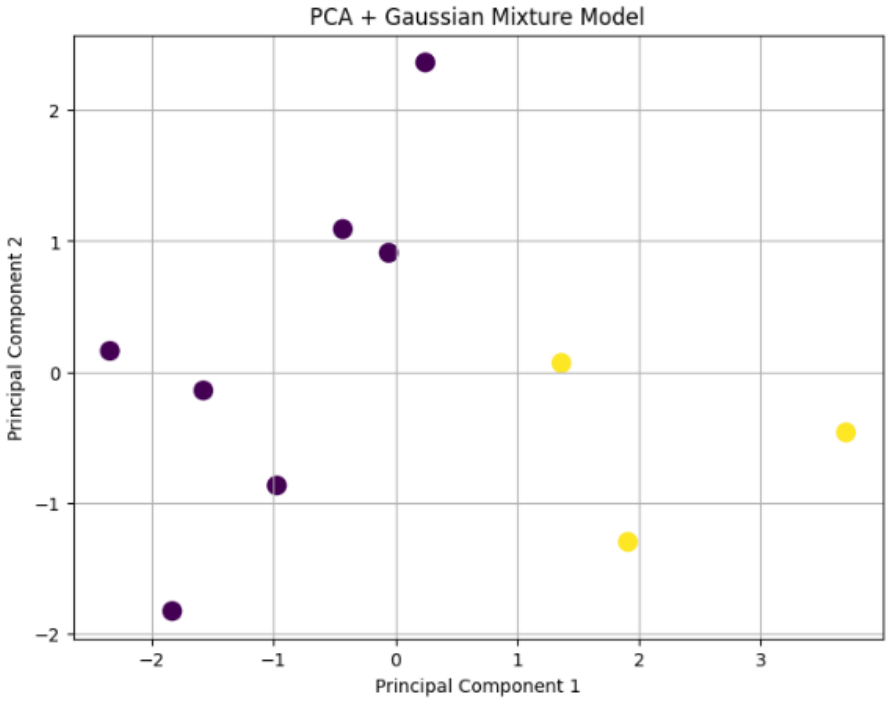
```
ICA Results:
      IC1    IC2
0 -0.376  0.683
1  1.689 -0.807
2 -0.344 -1.505
3 -1.163  1.662
4 -1.574 -1.411
5 -0.275  0.925
6  0.852 -0.353
7 -0.685 -0.345
8  1.056  0.798
9  0.821  0.354
```

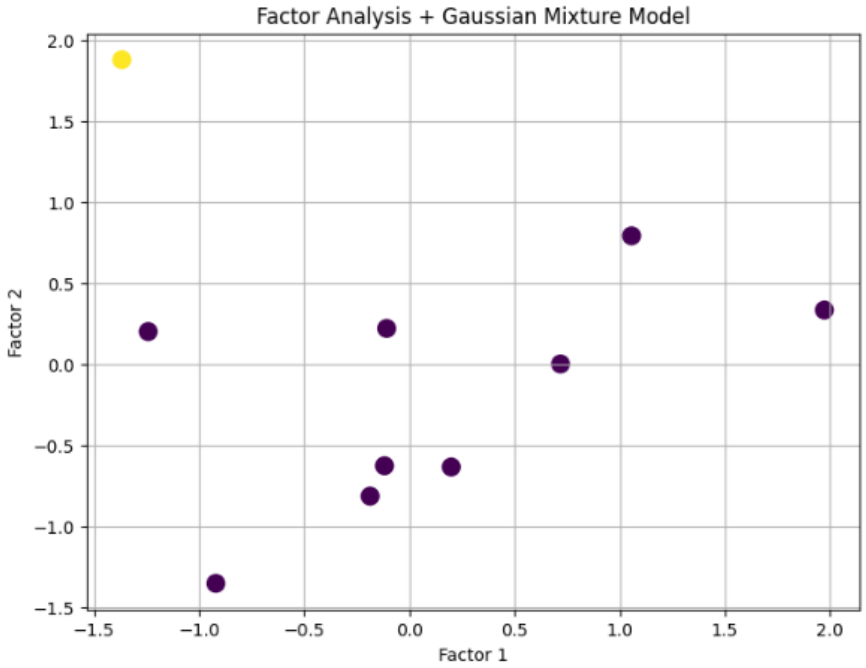
Important: ICA components have arbitrary sign and ordering. Therefore, the exact numerical values may change between implementations while representing equivalent independent components.

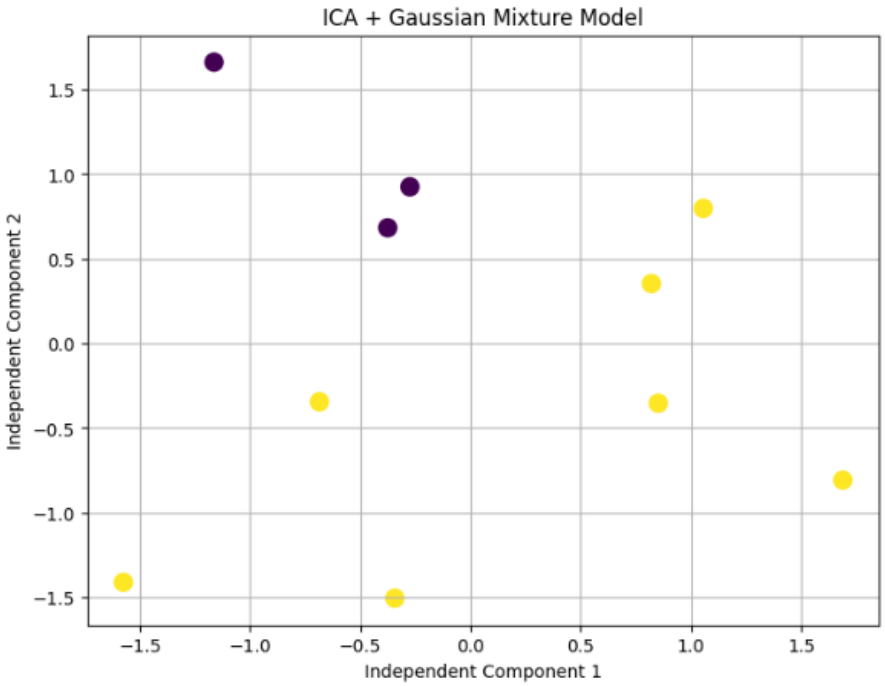
GMM Results:

Using two Gaussian components, the program performs clustering on the reduced datasets.

```
GMM Clusters using PCA:
[0 0 1 0 1 0 0 1 0 0]

GMM Clusters using Factor Analysis:
[0 1 0 0 0 0 0 0 0 0]

GMM Clusters using ICA:
[0 1 1 0 1 0 1 1 1 1]

Model Comparison using AIC and BIC
PCA + GMM: AIC = 84.03 BIC = 87.359
Factor Analysis + GMM: AIC = 43.463 BIC = 46.792
ICA + GMM: AIC = 68.517 BIC = 71.846
```







Final Cluster Comparison:

	Sample	PCA_Cluster	FA_Cluster	ICA_Cluster
0	1	0	0	0
1	2	0	1	1
2	3	1	0	1
3	4	0	0	0
4	5	1	0	1
5	6	0	0	0
6	7	0	0	1
7	8	1	0	1
8	9	0	0	1
9	10	0	0	1

AIC and BIC Comparison:

The program also calculates:

- AIC – Akaike Information Criterion
- BIC – Bayesian Information Criterion

For the given implementation, representative values are:

Method	**AIC**	**BIC**
PCA + GMM	84.030	87.359
Factor Analysis + GMM	43.463	46.792
ICA + GMM	68.517	71.846

Lower AIC and BIC values generally indicate a better balance between model fit and model complexity when comparing models fitted to the same type of observations/likelihood framework.

Therefore, these values should not be interpreted as proving that Factor Analysis is universally superior to PCA or ICA; they mainly provide a model-selection measure for the particular transformed representations.

Comparison of PCA, Factor Analysis and ICA:

Feature	**PCA**	**Factor Analysis**	**ICA**
Main purpose	Variance reduction	Find latent factors	Find independent components
Components	Principal components	Latent factors	Independent components
Components uncorrelated?	Yes	Model-dependent	Yes in the idealized formulation
Components independent?	No	No	Attempts to make them independent
Main objective	Maximum variance	Explain covariance using latent factors	Statistical independence
Useful for visualization	Yes	Yes	Yes

Useful for source separation	Limited	Limited	Strong
Interpretation	Variance-based	Latent-variable based	Independence-based

Difference Between PCA and Factor Analysis:

Although PCA and Factor Analysis are both dimensionality reduction methods, their objectives are different.

PCA:

PCA tries to find directions that maximize the variance of the observed data.

Factor Analysis:

Factor Analysis assumes that observed variables are generated by underlying latent factors.

Therefore:

PCA → variance representation

Factor Analysis → latent factor representation

Difference Between PCA and ICA:

PCA:

PCA creates components that are uncorrelated.

ICA:

ICA tries to identify statistically independent components.

Therefore, ICA goes beyond simply removing correlation.

For example:

- PCA asks: "Which directions explain the most variance?"
- ICA asks: "Which underlying components are as statistically independent as possible?"

Why GMM is Useful?

K-Means assigns each observation to one cluster based primarily on distance from centroids.

GMM is probabilistic.

For example, instead of saying:

Wine sample 1 definitely belongs to Cluster 0.

GMM can provide probabilities such as:

Cluster 0 = 0.85

Cluster 1 = 0.15

This is useful when cluster boundaries are uncertain or overlapping.

Business Interpretation:

The wine manufacturer can use the results to identify groups of wine samples with similar chemical characteristics.

For example, the manufacturer could identify:

Group 1:

Samples with similar alcohol, acidity, mineral and phenolic characteristics.

Group 2:

Samples with different chemical profiles.

This information could be useful for:

- Wine quality analysis
- Product categorization
- Production monitoring
- Chemical profiling
- Detecting unusual samples
- Developing product categories
- Further supervised learning

Overall Conclusion:

The wine dataset contains six chemical measurements. Because these measurements have different scales, standardization is performed before dimensionality reduction.

PCA reduces the six-dimensional data into two principal components while preserving important variance.

Factor Analysis represents the observations using latent factors, while ICA attempts to identify statistically independent components.

After dimensionality reduction, Gaussian Mixture Models are applied using the EM algorithm to identify probabilistic clusters.

The three methods produce different two-dimensional representations because they optimize different objectives:

- PCA focuses on maximum variance.
- Factor Analysis focuses on latent factors and covariance.
- ICA focuses on statistical independence.

Therefore, the most suitable dimensionality reduction method depends on the purpose of the analysis.